

Relatório IC - Estruturas de Dados Probabilísticas e Compactas

Bruno Rangel Pereira Tadim

2 de setembro de 2026

1 Introdução

Esse projeto de pesquisa visa estudar os fundamentos de estruturas probabilísticas e compactas, incluindo estruturas clássicas da área, suas respectivas vantagens e desvantagens, além de realizar implementações para experimentação empírica.

2 Entropia

A entropia de uma estrutura de dados é o limite mínimo de bits necessários para representar todos os possíveis estados que essa estrutura pode assumir.

A Teoria da Informação de Shannon nos diz que, se temos uma estrutura de dados com conjunto de estados U , onde cada estado $u \in U$ possui probabilidade de ocorrência $\Pr(u)$, dada por uma função $\Pr : U \rightarrow [0, 1]$ a entropia $\mathcal{H}(\Pr)$ é definida como:

$$\mathcal{H}(\Pr) = - \sum_{u \in U} \Pr(u) \log \Pr(u) = \sum_{u \in U} \Pr(u) \log \frac{1}{\Pr(u)}$$

Denominamos esse valor como entropia de Shannon.

Se a probabilidade de ocorrência for igual $\forall u \in U$, ou seja, $\Pr(u) = \frac{1}{|U|}$, então o cálculo de $\mathcal{H}(\Pr)$ será

$$\begin{aligned} \mathcal{H}(\Pr) &= - \sum_{u \in U} \frac{1}{|U|} \log \frac{1}{|U|} = \frac{1}{|U|} \left(- \sum_{u \in U} \log \frac{1}{|U|} \right) \\ &= \frac{1}{|U|} \left(-|U| \log \frac{1}{|U|} \right) = - \log \frac{1}{|U|} = \log |U| \end{aligned}$$

Chamamos o caso acima de *worst-entropy* e o denotamos $\mathcal{H}_{wc}$, pois o maior valor de $\mathcal{H}$ é obtido quando a distribuição de probabilidade é uniforme. Nesse caso, dizemos que o conjunto U é incompressível.

Para representarmos os elementos do conjunto U , atribuímos um código binário a cada um deles. Se todos os códigos tiverem o mesmo tamanho l , então o número de bits necessários será $\log |U| = \mathcal{H}_{wc}(U)$. Caso $l < \mathcal{H}_{wc}$, teríamos apenas $2^l < 2^{\mathcal{H}_{wc}} = 2^{\log |U|} = |U|$ códigos distintos, o que é insuficiente para representar todos os $|U|$ elementos.

No entanto, caso tenhamos informações sobre a distribuição de probabilidade Pr associada a U , e ela não for uniforme, podemos reduzir essa quantidade. Considere um conjunto $U = \{u_1, u_2, u_3, u_4\}$, com probabilidades $\{0.6, 0.2, 0.15, 0.05\}$, respectivamente. A entropia nesse caso é $\mathcal{H}(U) = 0.6 \cdot \log \frac{1}{0.6} + 0.2 \cdot \log \frac{1}{0.2} + 0.15 \cdot \log \frac{1}{0.15} + 0.05 \cdot \log \frac{1}{0.05} \approx 1.5332 < \log 4 = 2 = \mathcal{H}_{wc}(U)$.

Podemos atribuir o código 0 para u_1 , 10 para u_2 , 110 para u_3 e 111 para u_4 . Dessa forma, o tamanho médio dos códigos será $0.6 \cdot 1 + 0.2 \cdot 2 + 0.15 \cdot 3 + 0.05 \cdot 3 = 1.6$ bits, o que é próximo da entropia mínima e menor que o pior caso.

É importante que a codificação seja livre de prefixos, ou seja, nenhum código pode ser prefixo de outro, caso contrário, será impossível realizar a decodificação sem ambiguidades. Os códigos escolhidos acima foram gerados utilizando a **codificação de Huffman**.

2.1 Codificação de Huffman

A codificação de Huffman é um algoritmo que, dada uma distribuição de probabilidade $Pr : \Sigma \rightarrow [0.0, 1.0]$, retorna um código de prefixo que minimiza o tamanho médio dos códigos.

O algoritmo opera de forma gulosa e transforma um conjunto de $|\Sigma|$ nós em uma árvore binária. Inicialmente, temos um nó isolado para cada símbolo $s \in \Sigma$, contendo como valor a sua respectiva probabilidade $Pr(s)$. A cada passo, as duas raízes de menores probabilidades são mescladas, se tornando ambas filhas de uma nova raiz, que conterà como valor a soma dessas probabilidades.

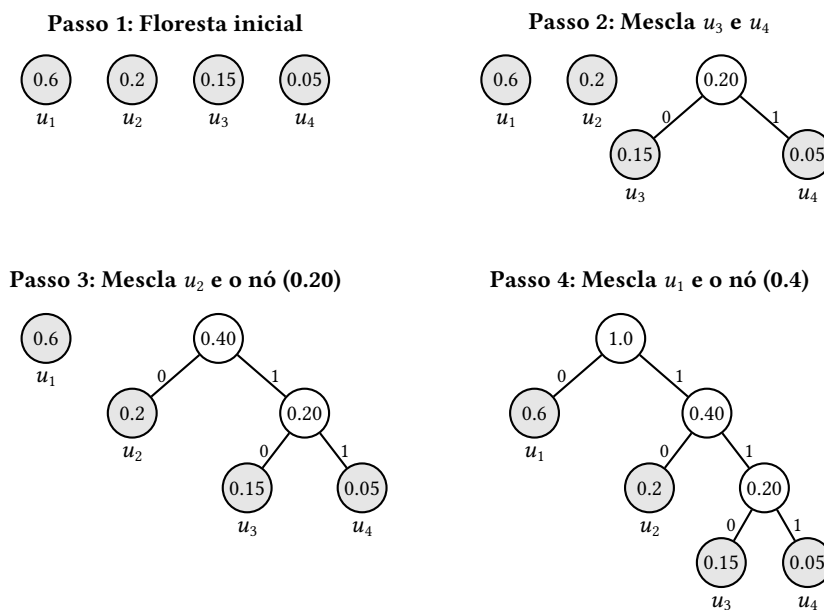


Figura 1: Execução passo a passo do algoritmo de Huffman para o conjunto U .

A árvore final gerada pelo algoritmo é denominada *árvore de Huffman*. Partindo da raiz da árvore, interpretamos ir para esquerda como 0 e ir para direita como 1. Dessa forma, o caminho da raiz da árvore até uma folha $s \in \Sigma$ representa o código referente a s , $C(s)$. A árvore de Huffman minimiza o tamanho médio dos códigos, dado por $\sum_{s \in \Sigma} \text{Pr}(s)l(s)$.

Ao implementarmos o algoritmo, podemos inserir os elementos em uma lista de prioridade (*Min-Heap*), ordenada pelas probabilidades. A cada iteração, extraímos os dois nós de peso mínimo, criamos um novo nó interno com a soma de suas probabilidades e o reinserimos na fila. Como cada operação de extração e inserção na fila de prioridade tem custo $\mathcal{O}(\log |\Sigma|)$ e o processo se repete até restar apenas um nó, a complexidade de tempo total do algoritmo é $\mathcal{O}(|\Sigma| \log |\Sigma|)$.

2.2 Codificação Aritmética

Diferentemente da codificação de Huffman, a codificação aritmética não se restringe a atribuição de códigos discretos a cada elemento, e sim utiliza a própria probabilidade de ocorrência de cada $s \in \Sigma$. Dessa forma, como não temos códigos discretos para cada elemento, calculamos um valor entre $[0, 1]$ para a mensagem toda.

3 Referências